

JavaScript

IF3110 – Web-based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Introduction

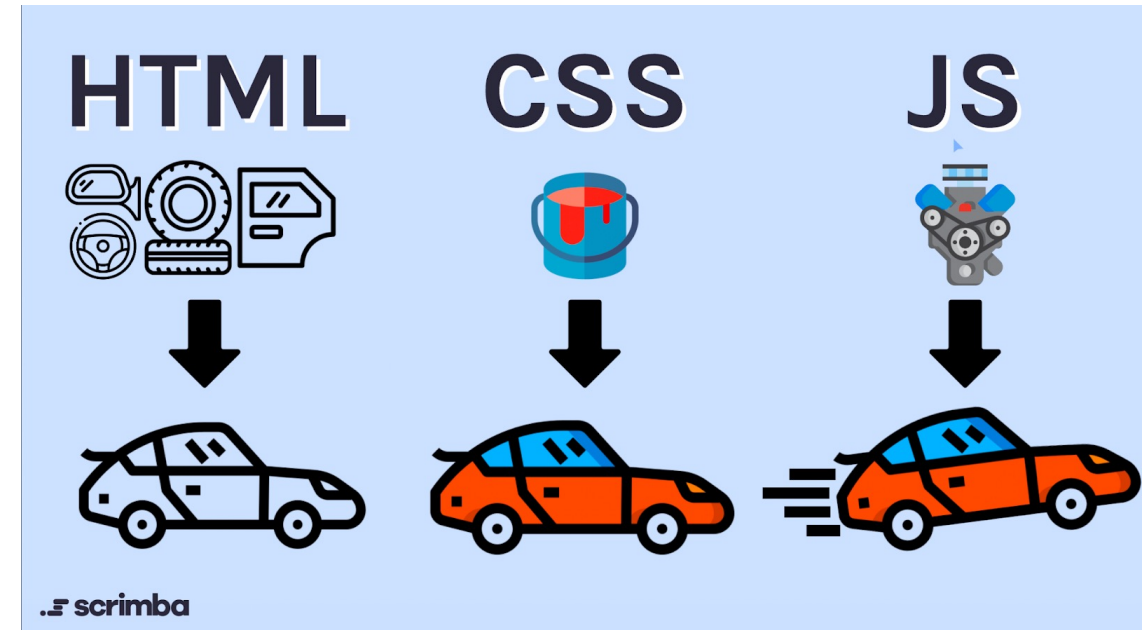
- Interpreter based, used to be only running on the browser – run on server-side (node.js) or in a JVM (rhino)
- JavaScript has similar syntax to Java, but it is **not** based on it
- Dynamic typing and supports duck typing
- Objects as general containers
- Function is the first-class (Lambda/closure)
- Linkage via global variables
- Prototypes over class-inheritance (class keyword is syntactic sugar to prototypes)
- Formalized & standardised as **ECMAScript** (**ECMA-262** and ISO/IEC 16262)

Introduction

- Javascript runs in browser:
 - Core syntax + DOM
- Has various implementation variant (the JavaScript Engine)
 - JavaScript Engine -> a special program embeded in a browser to execute JavaScript code. This program implements ECMAScript standard.
 - V8 -> Chrome, Edge, Opera, **Node.js (not a browser)**
 - SpiderMonkey -> Mozilla Firefox
 - JavaScriptCore -> WebKit/Safari
- support provided for DOM
 - The DOM is not, however, typically provided by the JavaScript engine but instead by a browser

Why do we need JS?

- HTML + CSS alone cannot handle **dynamic logic and complex user interaction**. HTML defines the structure of webpage. CSS defines only how the page looks.
- JavaScript allows a webpage to **respond to users, change content, process data, communicate with servers, and make decisions while the page is running**.

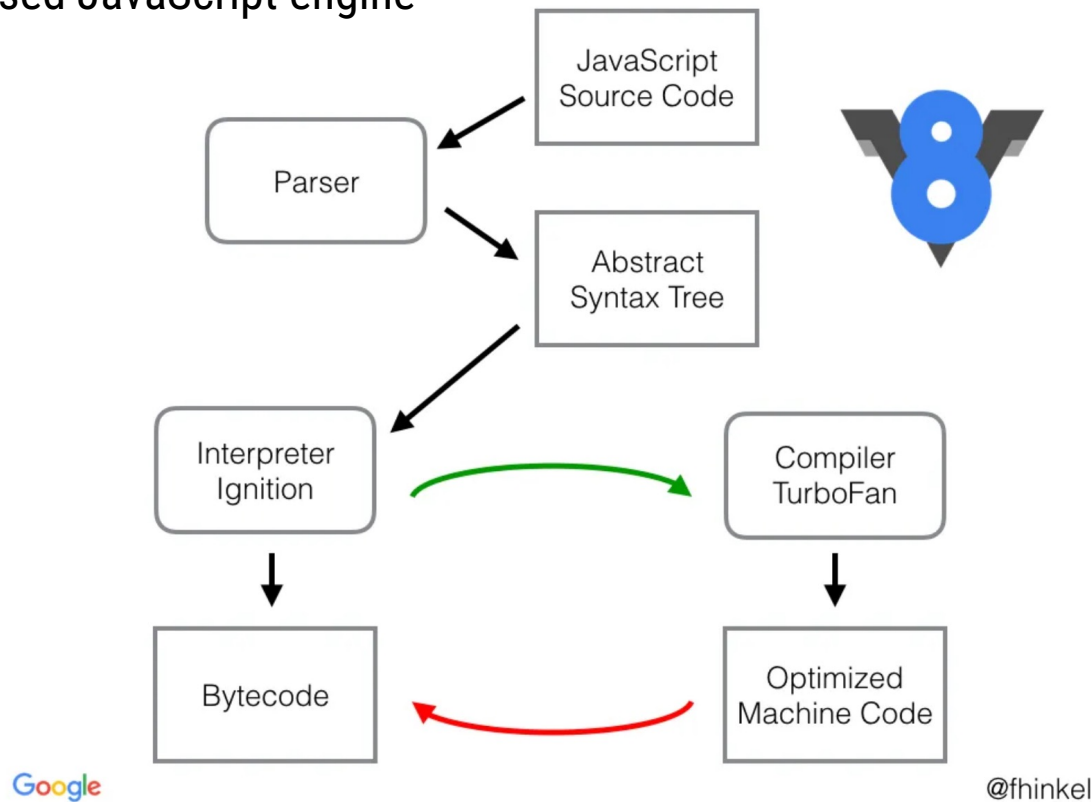


Source: <https://scrimba.com/articles/html-css-javascript/>

Inside the JavaScript Engine

V8 from Google is the most used JavaScript engine

written in C++



Courtesy: Franziska Hinkelmann's talk on V8

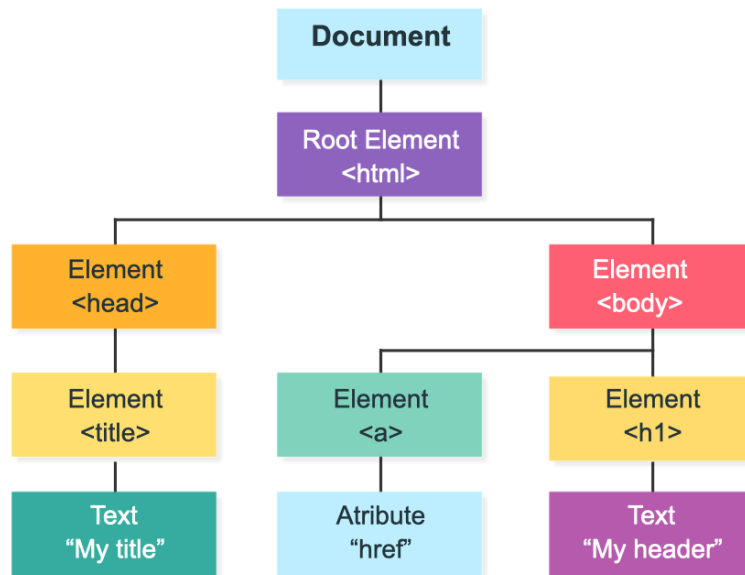
- V8 has had an interpreter as its first execution tier. This interpreter "compiles" JavaScript "just in time/JIT" to bytecode (which is then interpreted).
- JavaScript != Java, and it is not came from java.
- JavaScript run in Browser and Server-Side environment (e.g. Node.js)

How JavaScript works in Web

- Locating JavaScript in the HTML Document
 - internal HTML document
 - Standar HTML 4.01
`<script type="text/javascript">`
...statement...
`</script>`
 - Old Tag
`<script language="JavaScript">`
...statement...
`</script>`
 - file external (recommended)
 - Script JavaScript dituliskan pada file tersendiri (ekstensi file .js)
 - Pemanggilan file JavaScript:
`<script src="namafile.js"></script>`
- **Tips:** Place js script at the end of a html, to improve the performance

HTML Document Object Model and the JavaScript Code

- We also can run script inside console devtools in browser, because it ran V8 engine (e.g. Chrome)



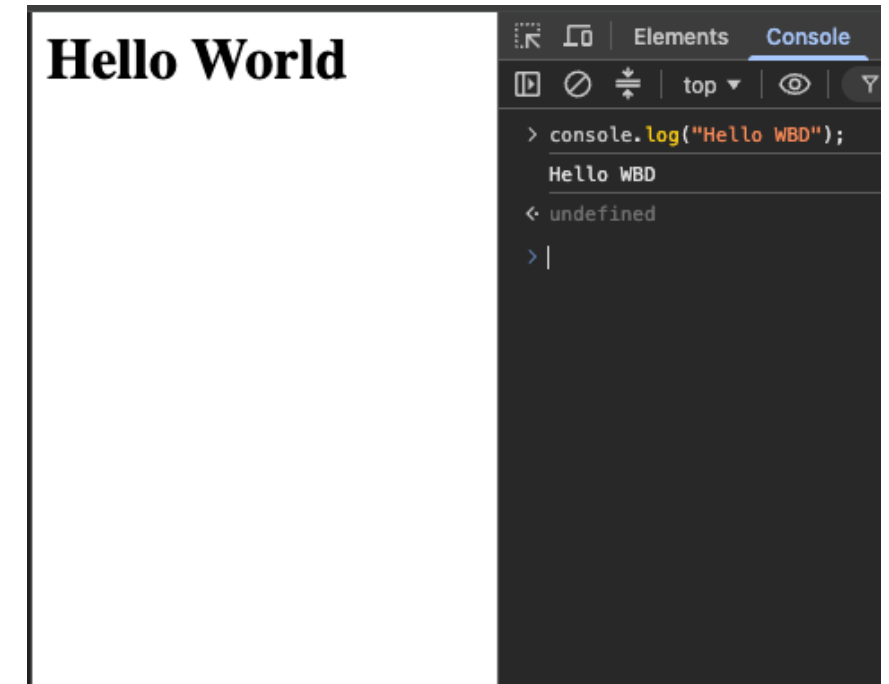
```
<html>
<body>

<h1 id="demo"></h1>

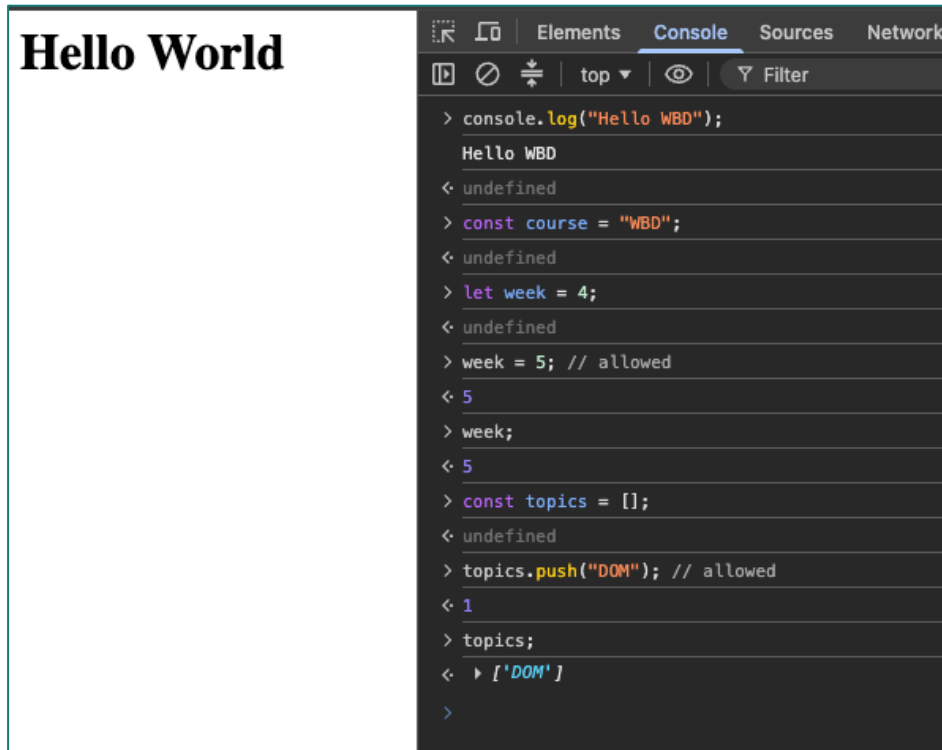
<script>
// Access a paragraph Element
const myPara =
document.getElementById("demo");

// Change the content of the Element
myPara.innerHTML = "Hello World";
</script>

</body>
</html>
```



JavaScript Variables.



The screenshot shows a web browser window with the text "Hello World" displayed. To the right, the browser's developer console is open, showing the following code and output:

```
> console.log("Hello WBD");  
Hello WBD  
< undefined  
> const course = "WBD";  
< undefined  
> let week = 4;  
< undefined  
> week = 5; // allowed  
< 5  
> week;  
< 5  
> const topics = [];  
< undefined  
> topics.push("DOM"); // allowed  
< 1  
> topics;  
< ▶ ['DOM']  
>
```

- Case sensitive
- Declare by **var**, **const**, or **let** (ES6, 2015)
 - `var i; var x = 2;`
 - `let y = 3; // having a block scope, and cannot be redeclared`
 - `const z = 4; // having a block scope, and cannot be redeclared & reassigned`
- Variable without a declaration, will be declared automatically as a global variable
 - `a = 5;`
- Scope: global dan local. local used when variable declared with in a function with `var`
- Convention: variable name, parameter, member, function starts with lowercase. Constructor starts with uppercase

Hoisting

- **Hoisting** refers to the process whereby the interpreter appears to move the *declaration* of functions, variables (declared using `var` keyword) to the top of their scope, prior to execution of the code.
- Behaviour:
 - Able to use a variable's value in its scope before the line it is declared.
 - Able to reference a variables in its scope without throwing error (the value is undefined)
- To prevent this, declare variable using `let` or `const` keyword.

How the code written:

```
console.log(x); // undefined  
var x = "Hello, World";
```



How actually executed:

```
var x;  
console.log(x); // undefined  
x = "Hello, World";
```

JavaScript Data Type

- Number : All numbers are represented in floating-point 64-bit IEEE-754 (double), but can be operated in integer
 - 0, 3, 10000000, 0xF3, 031,
 - 3.14, 1.23e10, 3.14E-14
 - NaN, Not a Number, every operations with NaN will become NaN
- String
 - Immutable
 - Some basic operations:
s = "hello"
s.length, s.charAt(2), sub = s.substring(2, 3)
i = s.indexOf('e')
"hello, world".replace("hello", "goodbye")
"hello".toUpperCase()
- Boolean :
 - true atau false
 - Boolean(value): cast some value into boolean. Alternative: use !!
 - Falsy: 0, **false**, **null**, **undefined**, "" (empty string), **NaN**
 - Truthy: other than falsy (including "null", "0", "false")

JavaScript Data Type

- Object: e.g.:
 - Function
 - Array
 - Date
 - Regexp, etc
- Null :
 - A value that isn't anything,
- Undefined
 - A value that isn't even that
 - The default value for variables and parameters
 - The value of missing members in objects
 - Undefined !== Null, it is different.
 - `typeof undefined`; → 'undefined'
 - `typeof null`; → 'object'

Hello World



```
> let wbd;  
< undefined  
> let sbd = null;  
< undefined  
> wbd !== sbd;  
< true  
> wbd  
< undefined  
> sbd  
< null  
> typeof undefined;  
< 'undefined'  
> typeof null;  
< 'object'  
> typeof [];  
< 'object'  
>
```

Operators Arithmetic, Assignment, Comparison

Math object provides a collection of standard function in arithmetic

abs	absolute value
floor	integer
log	logarithm
max	maximum
pow	raise to a power
random	random number
round	nearest integer
sin	sine
sqrt	square root

Operator	Usage	Example	Result
+	addition	3+4	7
-	subtraction	4-3	1
*	multiplication	4*3	12
/	division	4/3	1.33333333
%	modulus	4 % 3	1
++	increment	x=5 x++	x=6
--	decrement	x=5 x--	x=4

operator	example	explanation
=	a = b	assignment b to a
+=	a += b	a = a+b
-=	a -= b	a = a-b
*=	a *= b	a = a*b
/=	a /= b	a = a/b
%=	a %= b	a = a%b

Comparative Operator, Logic Operator

operator	example	explanation
=	a = b	assignment b to a
+=	a += b	a = a+b
-=	a -= b	a = a-b
*=	a *= b	a = a*b
/=	a /= b	a = a/b
%=	a %= b	a = a%b

operator	usage	example
&&	and	x = 6 y = 3 (x < 7 && y < 4)
	or	x = 6 y = 3 (x < 7 y < 2)
!	not	x = 6 y = 3 x != y

Loosely and Dynamically Typed

Loosely typed

- Any of these types can be stored in a variable, or passed as a parameter to any function.
- The language is not "untyped".
- Conversion of types happen automatically.

Dynamically typed

- Types are not required in advance.
- Types are not checked at compile-time.

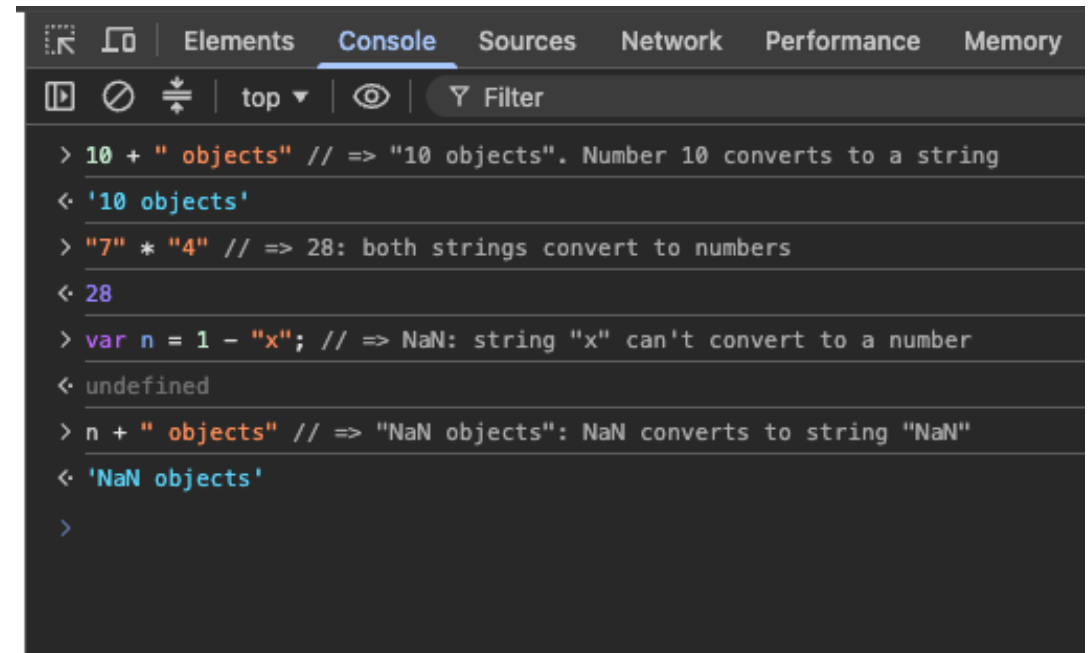
Type Conversion

`10 + " objects" // => "10 objects".`
Number 10 converts to a string

`"7" * "4" // => 28: both strings`
convert to numbers

`var n = 1 - "x"; // => NaN: string "x"`
can't convert to a number

`n + " objects" // => "NaN objects": NaN`
converts to string "NaN"



The screenshot shows a browser's developer console with the 'Console' tab selected. It displays the following JavaScript code and its output:

```
> 10 + " objects" // => "10 objects". Number 10 converts to a string
< '10 objects'
> "7" * "4" // => 28: both strings convert to numbers
< 28
> var n = 1 - "x"; // => NaN: string "x" can't convert to a number
< undefined
> n + " objects" // => "NaN objects": NaN converts to string "NaN"
< 'NaN objects'
>
```

Type Coercion

Equal and not equal operators

These operators can do type correction

It is better to use `===` (strict equality operator) and `!==`, which do not do type correction.

Examples:

```
"5" == 5    // true, because of type coercion
```

```
"5" === 5   // false, no type coercion occurs
```

For statement

Iterate through all of the elements of an array:

```
for (var i = 0; i < array.length; i += 1)
{
    // within the loop,
    // i is the index of the current
member    // array[i] is the current
          element
}
```

Iterate through all of the members of an object:

```
for (var name in object) {
    if (object.hasOwnProperty(name)) {
        // within the loop,
        // name is the key of current
member    // object[name] is the current
          value
    }
}
```

Switch statement

- Multiway branch
- The switch value does not need to a number. It can be a string.
- The case values can be expressions.

```
switch (expression) {  
  
    case ';' :  
  
    case ',' :  
  
    case '.' :  
  
        punctuation() ;  
  
        break ;  
  
    default :  
  
        noneOfTheAbove () ;  
  
}
```

Throw statement, try statement

```
throw new Error(reason);

throw {
  name: exceptionName,
  message: reason
};
```

```
try {
  ...
} catch (e) {
  switch (e.name) {
    case 'Error':
      ...
      break;
    default:
      throw e;
  }
}
```

The JavaScript implementation can produce these exception names:

'Error'

'EvalError'

'RangeError'

'SyntaxError'

'TypeError'

'URIError'

Function

- mechanism to structure a program
 - modular
 - reusable
- kind
 - built-in
 - user defined
- function can be
 - have parameters
 - return a value

syntax

```
function namaFungsi ([parameter]){  
    ... statements  
  
    [return value]  
}
```

function can be defined within another function

Function

- mechanism to structure a program
 - modular
 - reusable
- kind
 - built-in
 - user defined
- function can be
 - have parameters
 - return a value

syntax

```
function namaFungsi ([parameter]){  
    ... statements  
  
    [return value]  
}
```

function can be defined within another function

Function

- typical definition

```
function f(x, y) { return x*y; }
```

- constructor Function()

```
var f = new Function("x", "y",  
"return x*y;");
```

- function literal

```
var f = function(x, y) { return  
x*y; }
```

- arrow function

```
var f = (x, y) => return x*y;
```

function can be treated as data, stored in variable

```
function square(x) { return x*x; }  
var b = square;  
var c = b( 5 );
```

function can have parameter/argument

```
function square(x) { return x*x; }
```

argument can be access from the function w/ name or object arguments

```
function square(x) { return  
arguments[0]*x; }
```

argument checking/verification in Javascript done at runtime

Closure

A closure is the combination of a function bundled together (enclosed) with references to its surrounding state (the lexical environment). In other words, **a closure gives a function access to its outer scope**. In JavaScript, closures are created every time a function is created, at function creation time.

```
function makeFunc() {  
  const name = "Mozilla";  
  function displayName() {  
    console.log(name);  
  }  
  return displayName;  
}  
  
const myFunc = makeFunc();  
myFunc();
```

```
function makeAdder(x) {  
  return function (y) {  
    return x + y;  
  };  
}  
  
const add5 = makeAdder(5);  
const add10 = makeAdder(10);  
  
console.log(add5(2)); // 7  
console.log(add10(2)); // 12
```

Array & Its Method.

- **by Array constructor**

```
var a = new Array();  
var a = new Array( 1, 2,  
"tiga" );
```

- **by literal**

```
var a = [ 1, 2, "tiga" ];
```

- **array access**

```
a[0] = 1;  
a[7] = 8;
```

- **length of array**

```
a.length
```

join()

convert array values into string

reverse()

revers the value of array

sort()

sort the array values alphabetically

concat()

combing array values with the value of parameter

slice()

slice the value of array array

splice()

remove and add the value of array

push(), pop(), shift(), unshift()

Array Example

```
<html>
<script type="text/javascript">
//cara 1 pendefinisian array
mhs = new Array();
mhs[0] = "Bevin";
mhs[1] = "Andini";
mhs[2] = "Citra";
//cara 2 pendefinisian array
mhs = new Array("Bevin", "Andini", "Citra");
//cara 3 pendefinisian array
mhs = ["Bevin", "Andini", "Citra"];
//cara pengaksesan array
document.write("Mahasiswa pertama adalah "+mhs[0]+"<br>"); //Bevin
document.write("Mahasiswa terakhir adalah "+mhs[2]+"<br>"); //Citra
//cara pengaksesan array menggunakan loop
for (i=0; i<mhs.length; i++) {
    document.write(mhs[i] +"<br>");
}
//mengurutkan array
mhs.sort();
//menggabungkan array
document.write(mhs.join("-")); //Andini-Bevin-Citra
</script>
</html>
```

Object

- Java Script is prototype-based object language

- Object created by constructor

```
var now = new Date()
```

- Object can be created from literal

```
var circle = { x:0, y:0,  
radius:2 }
```

- `class` keyword is introduced in ES6 (2015)

- Attributed & methods access by . (dot)

```
var book = new Object();
```

```
book.title = "Javascript: The  
Definitive Guide"
```

```
book.author = "David  
Flanagan"
```

Object

- Property access using dot

```
object.property
```

- Or access as associative array

```
object["property"]
```

- property (attribute) can be enumerated w/ for loop

```
for( var name in obj ) {  
    document.write( name + "<BR>" );  
}
```

- property of an object can be removed

```
delete book.title;
```

Browser object

window

navigator

screen

history

location

Not standard, but provided in every browser

Browser object

Window object: global execution context,

```
var x = 5;
```

sama dengan

```
window.x = 5;
```

It refers to window browser (or frame) that renders the page

Document object: page that is being rendered

```
window.document
```

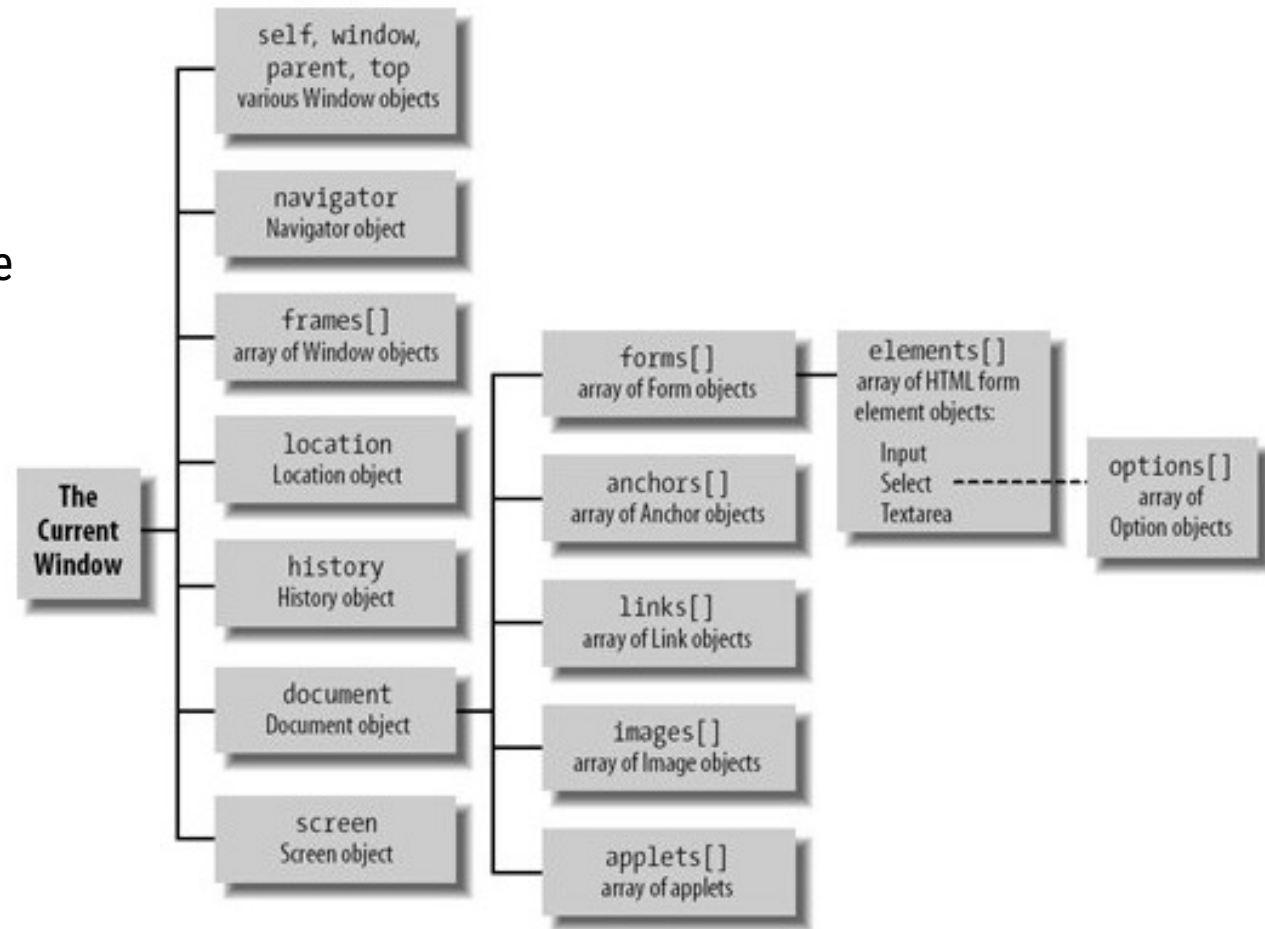
DOM (Document Object Model)

The **Document Object Model (DOM)** is a programming interface that represents a web page's HTML structure as a tree of objects in the browser's memory

Ex:

```
getElementById( id )  
getElementsByTagName( tag )  
createElement( tag )
```

HTMLDocument, derived from Document
body, forms, anchors, images, links
title, lastModified, referrers
etc
close(), open(), write()



Flanagan, David. Javascript: The Definitive Guide, 5th Ed.

Javascript as event handler

HTML Element can define to run script if an event occurs

onclick, onmousedown, onmouseup, onchange, onload

In-line HTML:

```
<input type="checkbox" name="options" value="giftwrap"  
onclick="giftwrap = this.checked;" >
```

About Event Listeners in JavaScript

Event Handler is function that handles when particular Event happened at specified target.

Object representing the event is passed as the first argument to the event handler. This event object either implements or is derived from the [Event](#) interface.

Used not only on UI, but also in workers, websocket, etc.

Two approaches:

- Onevent (e.g. onclick) method: simplicity and one to one.
- `addEventListener` and `removeEventListener`: Very Flexible and Specific, Allows multiple event listener attached to a target. Also enable Event Bubbling/Capture phase to be specified (for DOM event)

```
const btn = document.querySelector('button');

function greet(event) {
  console.log('greet:', event)
}

btn.onclick = greet;
```

```
const btn = document.querySelector('button');

function greet(event) {
  console.log('greet:', event)
}

btn.addEventListener('click', greet);
```

Example

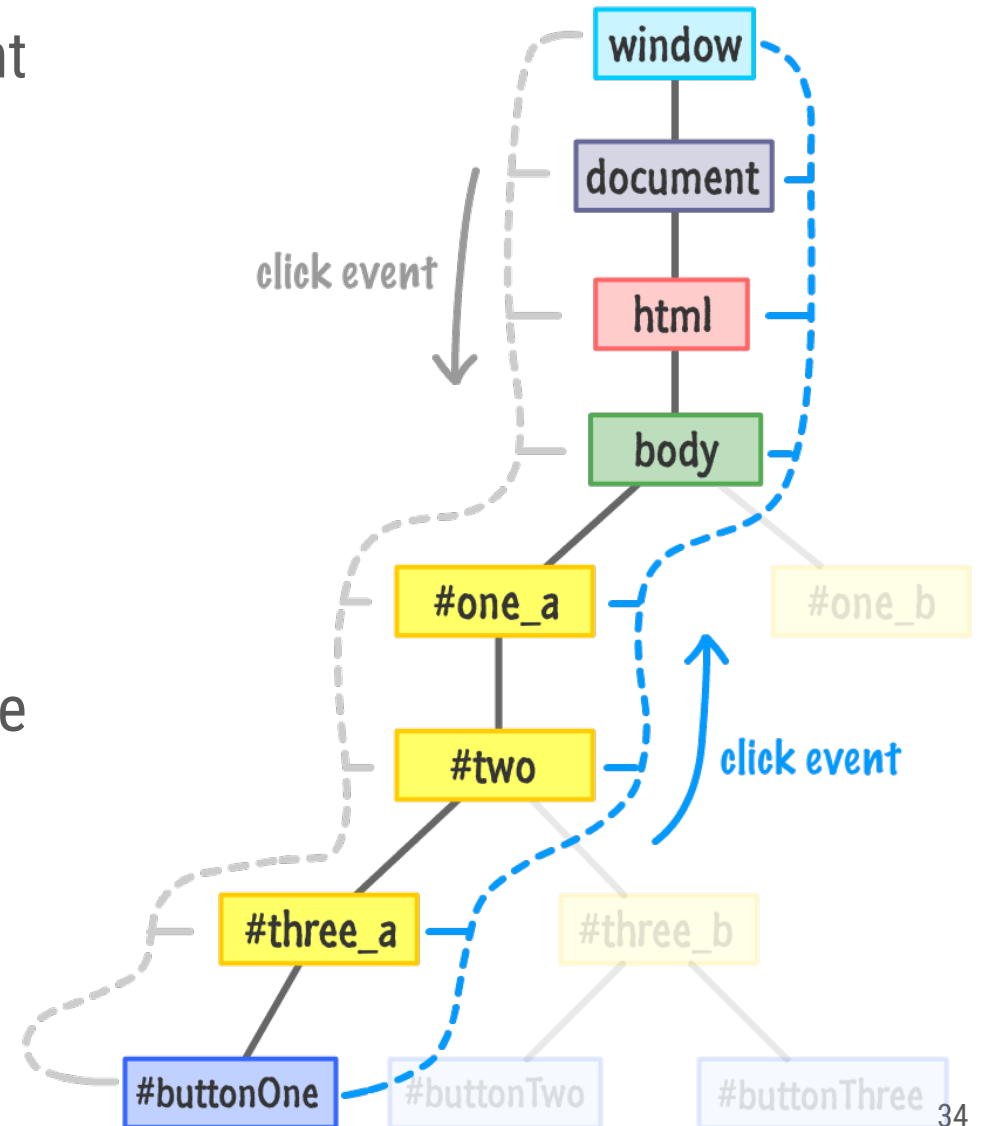
Script:

```
<script>
// Define a function to display function displayTime() {
// A script of js code the current time
var elt = document.getElementById("clock");
// Find element with id="clock"
}
window.onload = displayTime;
// Start displaying the time when document loads.
</script>
```

Event Bubble/Capture

The standard DOM Events describes 3 phases of event propagation:

- Capturing phase – the event goes down to the element.
- Target phase – the event reached the target element.
- Bubbling phase – the event bubbles up from the element.



Javascript Execution in HTML

Javascript is executed as soon as the element, containing script, has been processed by the browser.

To postponed the execution: use `defer` attributed,

OR

write a script as a function that is called by `onload` attributes from the body element

Javascript threading model

Javascript runs sequentially; with a single thread model.

On loading HTML document HTML, executing a script will suspend the loading process until the execution finishes

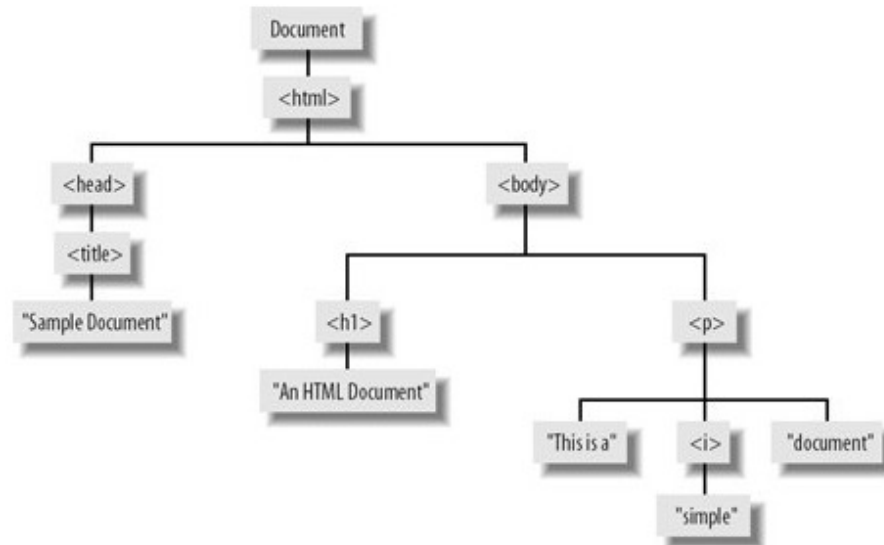
event-handler javascript SHOULD NOT be long, it creates a browser freeze.

W3C DOM

The **World Wide Web Consortium (W3C)** is the main international group that creates open standards and guidelines for the internet.

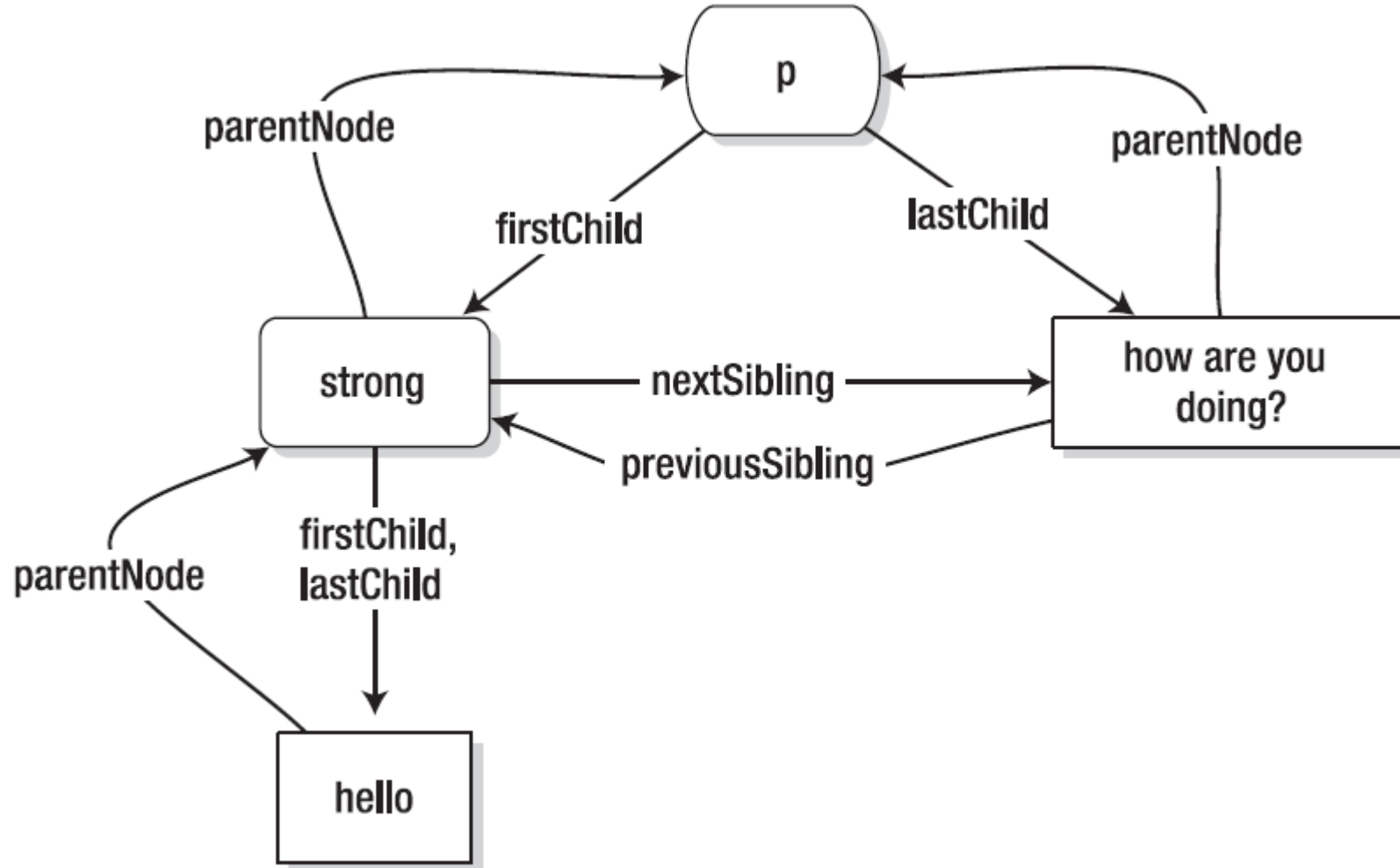
HTML Document is accessed by javascript via DOM

```
<html> <head> <title>Sample Document</title> </head> <body> <h1>An  
HTML Document</h1> <p>This is a <i>simple</i> document.</p> </html>
```



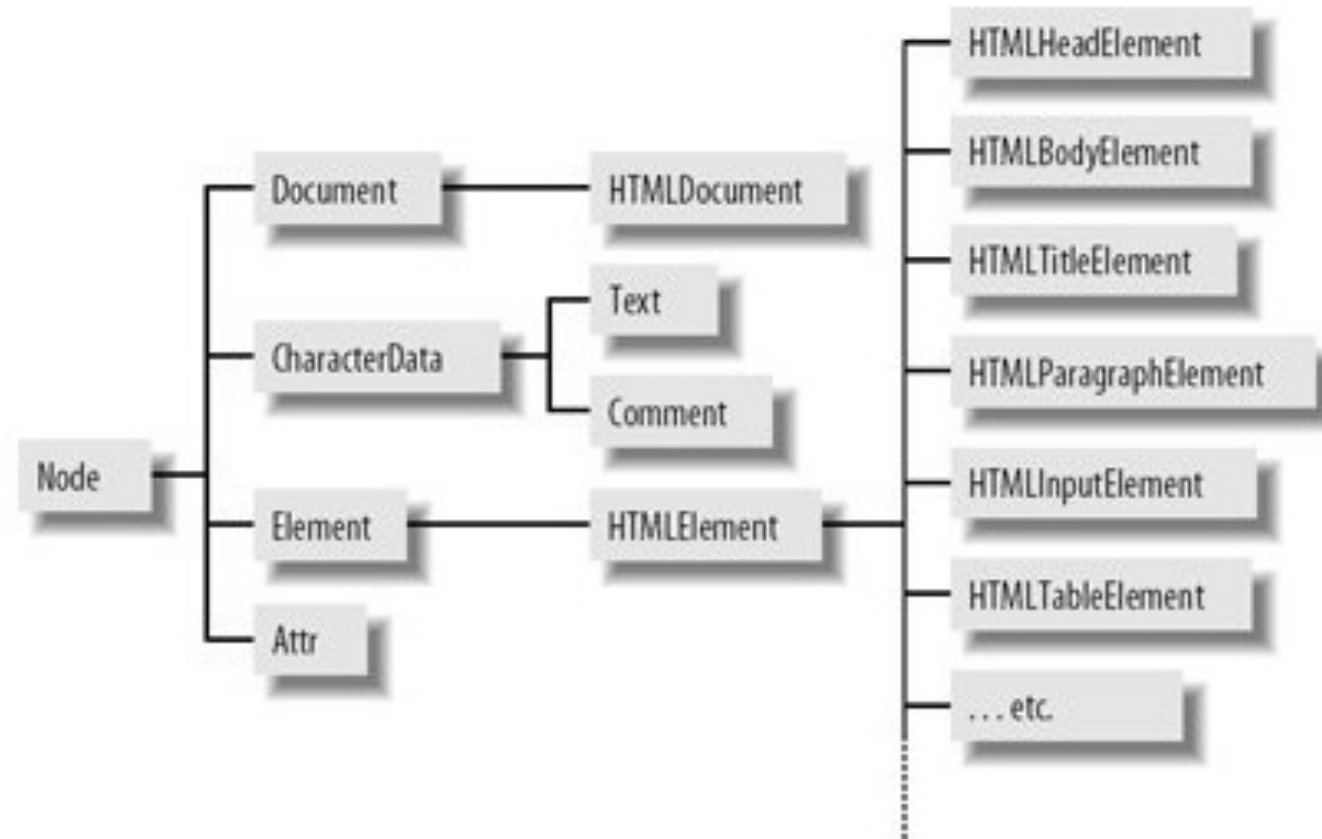
W3C DOM

`<p><strong>Hello</strong> how are you doing?</p>`



W3C DOM

HTML element in DOM is descendant of Node



W3C DOM

Accessing element:

```
document.getElementById(id_name);
```

return object Node object, if existed

```
Document.getElementsByTagName(tag_name);
```

return NodeList, can be accessed as array

```
document.getElementsByTagName("h1")[0]  
var li = document.getElementsByTagName("li");  
for ( var j = 0; j < li.length; j++ ) {  
    li[j].style.border = "1px solid #000";  
}
```

Memory Management

Regardless of the programming language, the memory life cycle is pretty much always the same:

1. Allocate the memory you need. In JS, It will automatically allocate memory when **values are initially declared**.
2. Use the allocated memory (read, write). This can be done by **reading or writing the value of a variable** or an object property or even passing an argument to a function.
3. Release the allocated memory when it is not needed anymore. JavaScript automatically frees it when **they are not used anymore** (garbage collection).

```
let text = "Hello"; // allocate memory for a string
console.log(text); // string is read

// string is deallocated
```

Garbage Collection

- The main concept that garbage collection algorithms rely on is the concept of reference. Within the context of memory management, an object is said to reference another object if the former has access to the latter (either implicitly or explicitly).
- **Reference-counting Algorithm:** determining whether or not an object is still needed to determining if an object still has any other objects referencing it (not used anymore in modern JavaScript engine)
- **Mark-and-sweep Algorithm:** reduces the definition of "an object is no longer needed" to "an object is unreachable" (all modern engines ship a mark-and-sweep garbage collector)

Memory Leak

- A **memory leak** occurs when a program keeps references to objects that are no longer needed, preventing the JavaScript engine's **garbage collector** from cleaning them up.
- Example:

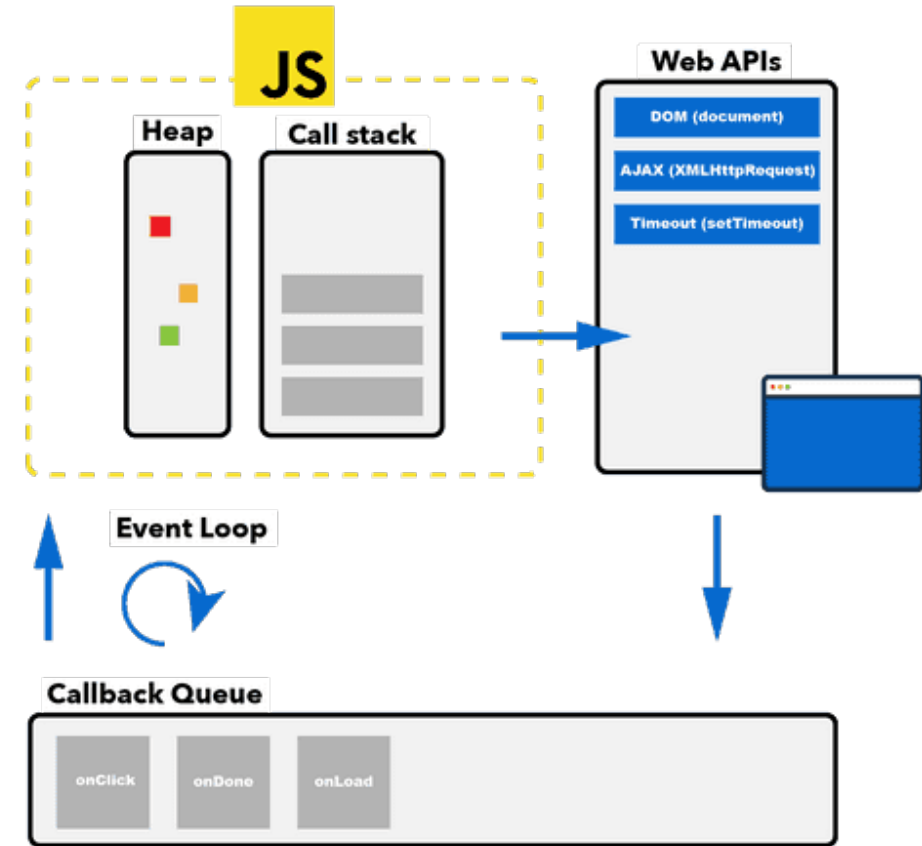
```
function leakyFunction() {  
    leakyVar = "I'm a global now"; // Oops! No 'var', 'let', or 'const'  
}
```

This function will create a global variable (`window.leakyVar`), which this variable will live until the page closed.

Javascript Runtime Model

JavaScript has a runtime model based on an **event loop**, which is responsible for executing the code, collecting and processing events, and executing queued sub-tasks. This model is quite different from models in other languages like C and Java.

JavaScript program is *single-threaded*. A thread is a sequence of instructions that a program follows. Because the program consists of a single thread, it can only do one thing at a time: so if it is waiting for our long-running synchronous call to return, it can't do anything else.



<https://felixgerschau.com/javascript-event-loop-call-stack/>

Runtime Concept: Call Stack

```
function foo(b) {  
  const a = 10;  
  return a + b + 11;  
}  
  
function bar(x) {  
  const y = 3;  
  return foo(x * y);  
}  
  
const baz = bar(7); // assigns 42 to baz
```

Function calls form a stack of frames.

Order of operations:

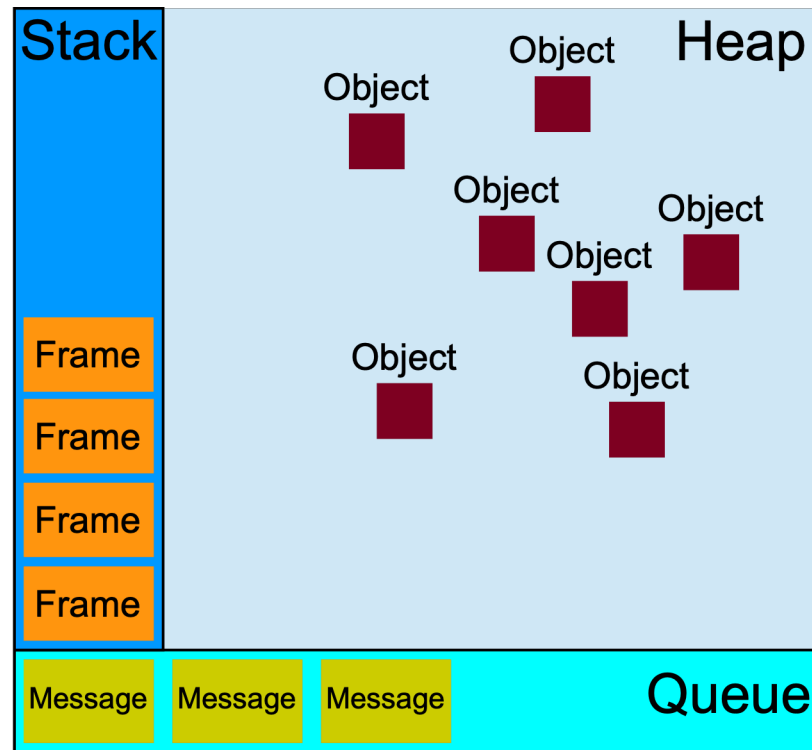
1. When calling **bar**, a first frame is created containing references to bar's arguments and local variables.
2. When bar calls **foo**, a second frame is created and pushed on top of the first one, containing references to foo's arguments and local variables.
3. When **foo** returns, the top frame element is popped out of the stack (leaving only bar's call frame).
4. When **bar** returns, the stack is empty.

See in action: <https://s.id/nu3r7>

Note that the arguments and local variables may continue to exist, as they are stored outside the stack

Runtime Concept: Heap

Objects are allocated in a heap which is just a name to denote a large (mostly unstructured) region of memory.



Runtime Concept: Queue

A JavaScript runtime uses a message queue, which is a list of messages to be processed. Each message has an associated function that gets called to handle the message.

Event loop processes have two queues:

- **Task Queue (Macrotask):** Contains scheduled units of work such as scripts, user events, timers, and other asynchronous callbacks. For example, executing button click event, `setTimeout()` / `setInterval()` callback.
- **Microtask Queue:** Contains short, high-priority follow-up work that runs after the current task finishes but before the next task begins. For example, Promise callbacks such as `.then()`, `.catch()`, and `.finally()`.

Event Loop

The event loop got its name because of how it's usually implemented, which usually resembles:

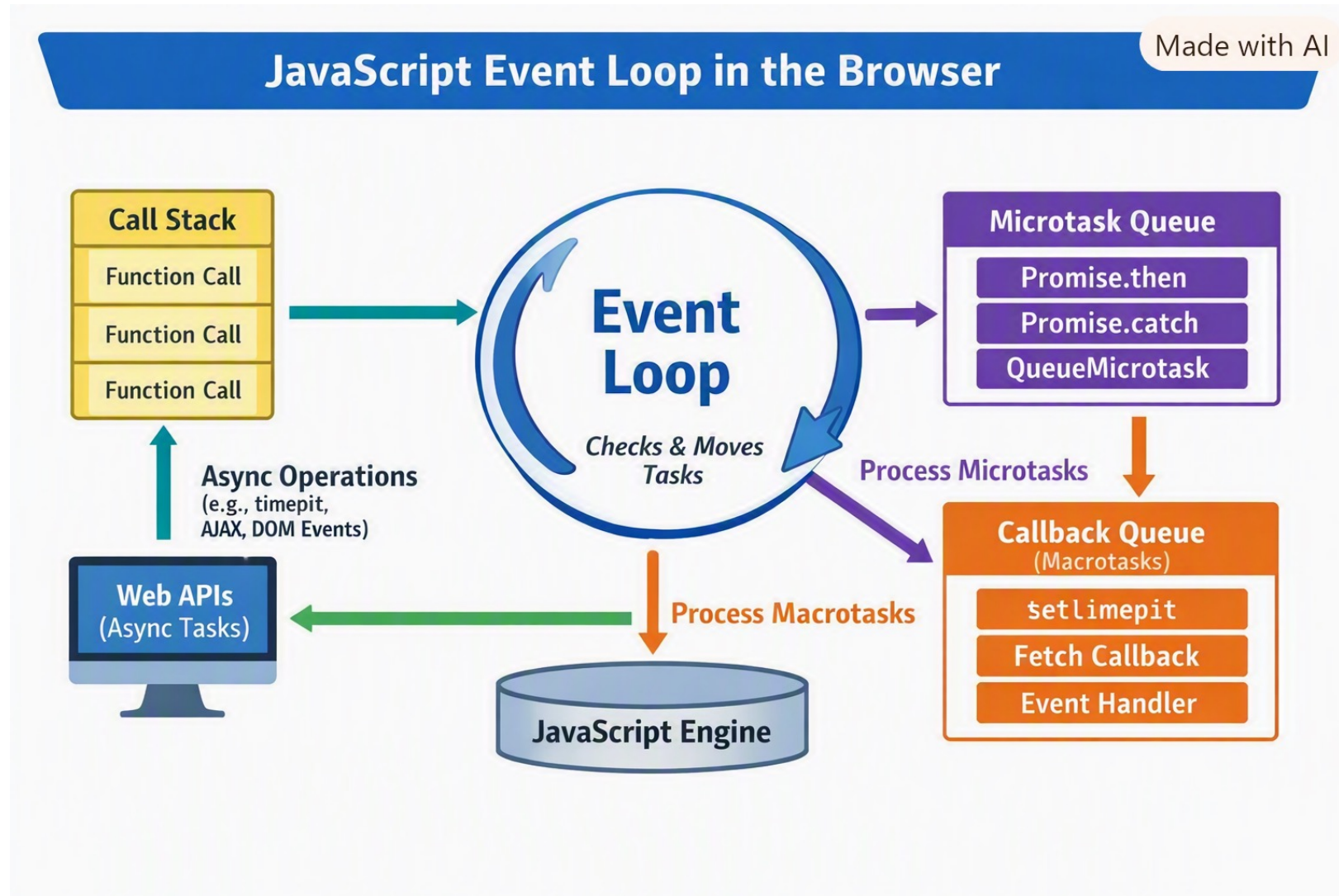
```
while (queue.waitForMessage()) {  
  queue.processNextMessage();  
}
```

Good Visualization of the Event Loop:

<https://www.youtube.com/watch?v=eiC58R16hb8>

A very interesting property of the event loop model is that JavaScript, unlike a lot of other languages, never blocks. Legacy exceptions exist like `alert` or synchronous XHR, but it is considered good practice to avoid them

Event Loop



Asynchronous JavaScript

```
const MAX_PRIME = 1000000;

function isPrime(n) {
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) {
      return false;
    }
  }
  return n > 1;
}

const random = (max) => Math.floor(Math.random() * max);

function generatePrimes(quota) {
  const primes = [];
  while (primes.length < quota) {
    const candidate = random(MAX_PRIME);
    if (isPrime(candidate)) {
      primes.push(candidate);
    }
  }
  return primes;
}
```

When multiple related things happen without any being dependent on the completion of previous happenings, they are asynchronous.

Look at the example, We have synchronous program that a very inefficient algorithm to generate multiple large prime numbers.

What if the synchronous function takes a long time?

A downside of the JavaScript runtime model is that **if a function takes too long to complete, the web application is unable to process user interactions like click or scroll.** The browser mitigates this with the "a script is taking too long to run" dialog.

Asynchronous JavaScript: Callback

A callback is just a function that's passed into another function, with the expectation that the callback will be called at the appropriate time.

However, callback-based code can get hard to understand when the callback itself has to call functions that accept a callback. This is a common situation if you need to perform some operation that breaks down into a series of asynchronous functions.

This is sometimes called "callback hell" or the "pyramid of doom".

```
function doStep1(init, callback) {
  const result = init + 1;
  callback(result);
}

function doStep2(init, callback) {
  const result = init + 2;
  callback(result);
}

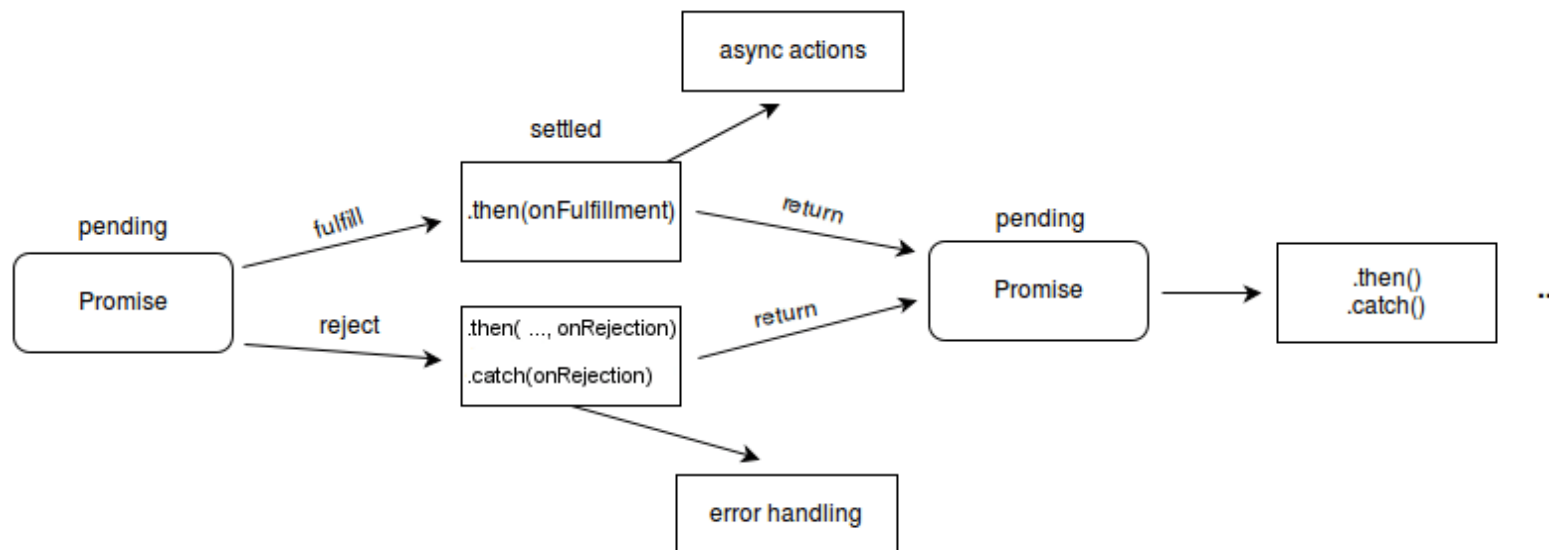
function doStep3(init, callback) {
  const result = init + 3;
  callback(result);
}

function doOperation() {
  doStep1(0, (result1) => {
    doStep2(result1, (result2) => {
      doStep3(result2, (result3) => {
        console.log(`result: ${result3}`);
      });
    });
  });
}

doOperation();
```

Asynchronous JavaScript: Promise

The Promise object represents the eventual completion (or failure) of an asynchronous operation and its resulting value. It allows you to associate handlers with an asynchronous action's eventual success value or failure reason



Promise State

- **pending**: initial state, neither fulfilled nor rejected.
- **fulfilled**: meaning that the operation was completed successfully.
- **rejected**: meaning that the operation failed.

Promise creation and chaining

Example below create and return new Promise Object. Then we handle or reject using then interface.

```
const myPromise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve("foo");  
  }, 300);  
});
```

```
myPromise  
  .then(handleFulfilledA, handleRejectedA)  
  .then(handleFulfilledB, handleRejectedB)  
  .then(handleFulfilledC, handleRejectedC);
```

The promise chain is what you need when your operation consists of several asynchronous functions, and you need each one to complete before starting the next one.

This special then() function returns another promise p. Depends on the return of value of the handler which may:

- returns a value: p gets **fulfilled** with the returned value as its value.
- doesn't return anything: p gets **fulfilled** with undefined as its value.
- throws an error: p gets **rejected** with the thrown error as its value.
- returns another pending promise: p is **pending** and becomes fulfilled/rejected with that promise's value as its value immediately after that promise becomes fulfilled/rejected.

Asynchronous JavaScript: Async Await

```
async function fetchProducts() {
  try {
    // after this line, our function will wait for the `fetch()` call to be settled
    // the `fetch()` call will either return a Response or throw an error
    const response = await fetch(
      "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-
store/products.json",
    );
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    // after this line, our function will wait for the `response.json()` call to be
    settled
    // the `response.json()` call will either return the parsed JSON object or throw
    an error
    const data = await response.json();
    console.log(data[0].name);
  } catch (error) {
    console.error(`Could not get products: ${error}`);
  }
}

fetchProducts();
```

The `async` keyword gives you a simpler way to work with asynchronous promise-based code. Adding `async` at the start of a function makes it an async function.

Inside an async function, you can use the `await` keyword before a call to a function that returns a promise.

The `async` and `await` keywords make it easier to build an operation from a series of consecutive asynchronous function calls, avoiding the need to create explicit promise chains, and allowing you to write code that looks just like synchronous code (a cleaner style).

TypeScript

What is TypeScript?:

Programming Language: A language that includes all the existing JavaScript syntax, plus new TypeScript-specific syntax for defining and using types

Type Checker: A program that takes in a set of files written in JavaScript and/or TypeScript, checks for incorrect syntaxes

Compiler (Transpiler) : A program that runs the type checker, reports any issues, then outputs the equivalent JavaScript code.

Language Service: A program that uses the type checker to tell editors such as VS Code how to provide helpful utilities to developers

TypeScript

- JS
 - Unconstrained
 - Checked at runtime
 - Errors surfaced at runtime (mostly)
- TS
 - Constrained or bound statically
 - Checked at compile time
 - Errors surfaced at compile time (mostly)

```
function squareOf(n) {  
    return n * n  
}  
  
squareOf(2)           // evaluates to 4  
squareOf('z')        // evaluates to NaN
```

```
function squareOf(n: number) {  
    return n * n  
}  
  
squareOf(2)           // evaluates to 4  
squareOf('z')        // Error TS2345: Argument of type '"z"' is not  
                        // parameter of type 'number'.
```